

GUÍA DE LABORATORIO: OPTIMIZACIÓN EN LA JERARQUÍA DE MEMORIA

Análisis Empírico de Líneas de Caché (64B), Registros de CPU y Paralelismo ILP en Linux

"El código de alto nivel se ejecuta sobre circuitos físicos: comprender el comportamiento de las líneas de caché y los registros del procesador diferencia a un codificador de un verdadero Ingeniero de Software."

1. OBJETIVOS DEL LABORATORIO

- Comprobar empíricamente la penalización temporal de un fallo de caché (**Cache Miss**) frente al acierto (**Cache Hit**).
- Comprender la estructura física de la **Línea de Caché de 64 Bytes** y su aprovechamiento mediante el orden contiguo (*Row-Major Order*).
- Manipular la retención de operandos en **Registros del CPU (FPU/ALU)** para evitar el cuello de botella del bus de memoria.
- Analizar el **Paralelismo a Nivel de Instrucción (ILP)** mediante desenrollado de bucles (*Loop Unrolling*).
- Obtener mediciones determinísticas, calcular el factor de aceleración (**Speedup**: $S = T_{base} / T_{opt}$) y el rendimiento en **GFLOPS**.

2. VERIFICACIÓN PREVIA DEL HARDWARE EN LINUX

Antes de compilar, abra una terminal en Linux y ejecute los siguientes comandos para conocer la topología de memoria y la línea de caché física de su microprocesador:

```
# 1. Consultar jerarquía y tamaños de memoria caché L1, L2 y L3
$ lscpu | grep -E "L1|L2|L3|Model name"

# 2. Consultar el tamaño exacto de la línea de caché de datos L1 (típicamente 64 Bytes)
$ getconf LEVEL1_DCACHE_LINESIZE

# 3. Inspeccionar el sysfs del kernel para confirmar el tamaño de bloque de coherencia
$ cat /sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size
```

Principio Físico: Una línea de **64 Bytes** almacena exactamente **16 números de punto flotante de 32 bits** (`sizeof(float) = 4 bytes`). Cuando la CPU solicita un elemento, el bus siempre transfiere la línea completa de 16 elementos contiguos.

3. ARQUITECTURA DEL ALGORITMO Y VARIANTES EXPERIMENTALES

El experimento evalúa el producto matricial $C = A \times B$ con dimensiones 1024×1024 (matriz de 4 MB; huella activa de 24 MB en RAM):

- Fase 1 (Naive i - j - k):** Algoritmo directo. En $B[k][j]$, el índice que salta en el bucle interno es k , brincando 4096 bytes en cada paso. Genera *Cache Misses* continuos y desaprovecha el 93.75% de cada transferencia de memoria.
- Fase 2 (Localidad Espacial i - k - j):** Se intercambian bucles. Al variar j , tanto $B[k][j]$ como $C[i][j]$ se acceden de manera contigua. Se produce 1 fallo de caché cada 16 lecturas (tasa de aciertos teórica $\geq 93.75\%$).
- Fase 3 (Localidad Temporal + Registros de CPU):** El factor $A[i][k]$ es constante durante todo el bucle j . Se mantiene en un registro físico de punto flotante (`register const float reg_a`) sin tocar la RAM.
- Fase 4 (Loop Unrolling 4x + ILP):** Se procesan 4 elementos por paso ($j, j+1, j+2, j+3$). Reduce la sobrecarga de saltos y permite que el procesador superescalar ejecute múltiples instrucciones en paralelo en sus pipelines.

4. CÓDIGO FUENTE OFICIAL DEL BENCHMARK EN ANSI C

Guarde el siguiente código en el archivo [benchmark_arquitectura.c](#) en su directorio de trabajo en Linux:

```
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

#define N 1024

static double medir_tiempo_segundos(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}

/* 1. NAIVE (i-j-k): Ineficiente, Cache Miss masivo */
void algoritmo_naive(const float *A, const float *B, float *C) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            float suma = 0.0f;
            for (int k = 0; k < N; k++) {
                suma += A[i * N + k] * B[k * N + j]; // Salto de fila en B: 4096 bytes
            }
            C[i * N + j] = suma;
        }
    }
}

/* 2. LOCALIDAD ESPACIAL (i-k-j): Aprovecha linea de cache de 64 bytes */
void algoritmo_localidad_espacial(const float *A, const float *B, float *C) {
    for (int idx = 0; idx < N * N; idx++) C[idx] = 0.0f;
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            float r = A[i * N + k];
            int fila_b = k * N, fila_c = i * N;
            for (int j = 0; j < N; j++) {
                C[fila_c + j] += r * B[fila_b + j]; // Acceso estrictamente contiguo
            }
        }
    }
}

/* 3. LOCALIDAD TEMPORAL + REGISTROS: Mantiene A[i][k] en registro del CPU */
void algoritmo_registros_cpu(const float *A, const float *B, float *C) {
    for (int idx = 0; idx < N * N; idx++) C[idx] = 0.0f;
    for (int i = 0; i < N; i++) {
        float *ptr_c = &C[i * N];
        for (int k = 0; k < N; k++) {
            register const float reg_a = A[i * N + k]; // En registro de la FPU
            const float *ptr_b = &B[k * N];
            for (int j = 0; j < N; j++) {
                ptr_c[j] += reg_a * ptr_b[j];
            }
        }
    }
}

/* 4. LOOP UNROLLING 4X + ILP: Paralelismo a nivel de instruccion */
void algoritmo_loop_unrolling(const float *A, const float *B, float *C) {
    for (int idx = 0; idx < N * N; idx++) C[idx] = 0.0f;
    for (int i = 0; i < N; i++) {
        float *ptr_c = &C[i * N];
        for (int k = 0; k < N; k++) {
            register const float reg_a = A[i * N + k];
            const float *ptr_b = &B[k * N];
            for (int j = 0; j < N; j += 4) {
                ptr_c[j] += reg_a * ptr_b[j];
                ptr_c[j + 1] += reg_a * ptr_b[j + 1];
                ptr_c[j + 2] += reg_a * ptr_b[j + 2];
                ptr_c[j + 3] += reg_a * ptr_b[j + 3];
            }
        }
    }
}
```

```

}

static double calcular_checksum(const float *mat) {
    double sum = 0.0;
    for (int i = 0; i < N * N; i++) sum += (double)mat[i];
    return sum;
}

int main(void) {
    size_t total = (size_t)N * N, bytes = total * sizeof(float);
    double gflops = (2.0 * (double)N * N * N) / 1e9;
    float *A = (float*)malloc(bytes), *B = (float*)malloc(bytes);
    float *C1 = (float*)malloc(bytes), *C2 = (float*)malloc(bytes);
    float *C3 = (float*)malloc(bytes), *C4 = (float*)malloc(bytes);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i * N + j] = (float)((i + j) % 50) * 0.02f + 1.0f;
            B[i * N + j] = (float)((i * 2 + j) % 50) * 0.02f + 0.5f;
        }
    }

    double t0, t1, t_naive, t_cache, t_reg, t_unroll;
    t0 = medir_tiempo_segundos(); algoritmo_naive(A, B, C1); t_naive = medir_tiempo_segundos() - t0;
    t0 = medir_tiempo_segundos(); algoritmo_localidad_espacial(A, B, C2); t_cache = medir_tiempo_segundos() - t0;
    t0 = medir_tiempo_segundos(); algoritmo_registros_cpu(A, B, C3); t_reg = medir_tiempo_segundos() - t0;
    t0 = medir_tiempo_segundos(); algoritmo_loop_unrolling(A, B, C4); t_unroll = medir_tiempo_segundos() - t0;

    printf("\n=== RESULTADOS DETERMINISTICOS (N=%d) ===\n", N);
    printf("1. Naive (i-j-k) : %7.4f s | %6.2f GFLOPS | Speedup: 1.00x\n", t_naive, gflops/t_naive);
    printf("2. Localidad Espacial (Cache) : %7.4f s | %6.2f GFLOPS | Speedup: %.2fx\n", t_cache, gflops/t_cache, t_naive/t_cache);
    printf("3. Registros de CPU : %7.4f s | %6.2f GFLOPS | Speedup: %.2fx\n", t_reg, gflops/t_reg, t_naive/t_reg);
    printf("4. Loop Unrolling 4x (ILP) : %7.4f s | %6.2f GFLOPS | Speedup: %.2fx\n", t_unroll, gflops/t_unroll, t_naive/t_unroll);

    double chk1 = calcular_checksum(C1), chk4 = calcular_checksum(C4);
    printf("[OK] Validacion de Checksum: %.4e (Error = %.4e)\n", chk1, fabs(chk1 - chk4));
    free(A); free(B); free(C1); free(C2); free(C3); free(C4);
    return 0;
}

```

5. COMPILACIÓN Y EJECUCIÓN PASO A PASO EN LINUX

Siga rigurosamente las siguientes instrucciones en su terminal Linux:

Paso 1: Compilación Determinística Oficial

```
$ gcc -Wall -Wextra -O1 benchmark_arquitectura.c -o benchmark_arquitectura -lm
```

¿Por qué el flag -O1? El nivel de optimización -O1 activa optimizaciones mecánicas básicas pero no reordena bucles automáticamente (a diferencia de -O3). Esto permite que el estudiante mida el impacto 100% real de la arquitectura del código que ha programado.

Paso 2: Ejecución Directa y Medición de Tiempos

```
$ ./benchmark_arquitectura
```

Paso 3: Análisis de Fallos de Caché con la Herramienta `perf` en Linux

Linux cuenta con la herramienta de perfilado de bajo nivel perf, que accede a los contadores de rendimiento de la CPU (PMU). Ejecute:

```
# Instalar perf en Ubuntu/Debian si no estuviera instalado:
$ sudo apt-get update && sudo apt-get install -y linux-tools-generic linux-tools-common

# Medir eventos de hardware en la ejecucion del benchmark:
$ perf stat -e L1-dcache-loads,L1-dcache-load-misses,cycles,instructions ./benchmark_arquitectura
```

Observe en el reporte de perf cómo la tasa de **L1-dcache-load-misses** cae drásticamente y el índice de **Instructions Per Cycle (IPC)** se incrementa.

6. PLANTILLA DE REGISTRO DE MÉTRICAS (PARA EL ESTUDIANTE)

Complete la siguiente tabla con los valores reales obtenidos en su computador:

Fase / Configuración	Tiempo Medido (s)	Rendimiento (GFLOPS)	Aceleración (Speedup)	Tasa Acierto Caché
1. Naive (i-j-k)	_____ s	_____ GFLOPS	1.00x (Base)	Baja (< 15%)
2. Localidad Espacial (i-k-j)	_____ s	_____ GFLOPS	_____ x	Alta (> 90%)
3. Uso de Registros CPU	_____ s	_____ GFLOPS	_____ x	Óptima
4. Loop Unrolling 4x (ILP)	_____ s	_____ GFLOPS	_____ x	Máxima

7. CUESTIONARIO DE ANÁLISIS CRÍTICO (ENTREGABLE DEL LABORATORIO)

Responda puntualmente con sustento de arquitectura de computadoras:

- Pregunta 1 (Líneas de Caché):** Si una línea de caché mide 64 bytes y cada float ocupa 4 bytes, ¿cuántos accesos a memoria consecutivos aprovechan una sola carga a la caché L1 en la Fase 2? Demuestre la fórmula teórica del *Hit Rate*.
- Pregunta 2 (Localidad Temporal y Registros):** En la Fase 3, ¿qué ventaja física representa almacenar reg_a en un registro de la FPU/ALU en lugar de releerlo desde la memoria en cada paso del bucle interior?
- Pregunta 3 (Loop Unrolling e ILP):** ¿Por qué el desenrollado de bucles en la Fase 4 reduce el tiempo de ejecución incluso cuando el número total de sumas y multiplicaciones matemáticas es exactamente el mismo?
- Pregunta 4 (Validación de Determinismo):** ¿Por qué el valor del *Checksum* matemático debe ser idéntico en las cuatro fases? ¿Qué indicaría si el Checksum de la Fase 4 difiere del de la Fase 1?
- Formato de Entrega:** Los estudiantes deben entregar un informe técnico en formato PDF o Markdown conteniendo: capturas de pantalla de la terminal Linux con la ejecución, la tabla de métricas completa y las respuestas al cuestionario.